

Chapter 2A:

Solving Problems by Searching (Part I)

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Problem Solving Agents
- Problem Solving Phases
- Problem Formulation
- Abstraction
- Standardized Problems
- Real-World Problems
- Search Algorithms
- Search Tree
- Frontier vs. Reached

Content

- Search Data Structures
- Loopy Paths and Redundant Paths
- Tree-Like Search Algorithm
- Graph Search Algorithm

Problem Solving Agents

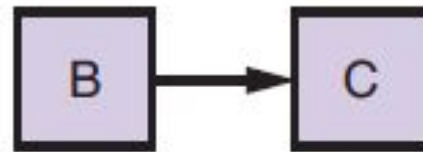
- **Problem solving agent**

- A type of **goal-based agent**
- Find a sequence of actions that achieves its goals

- **Environment states**

- **Atomic representation**

- Each state of the world is indivisible without any internal structure
- Example:



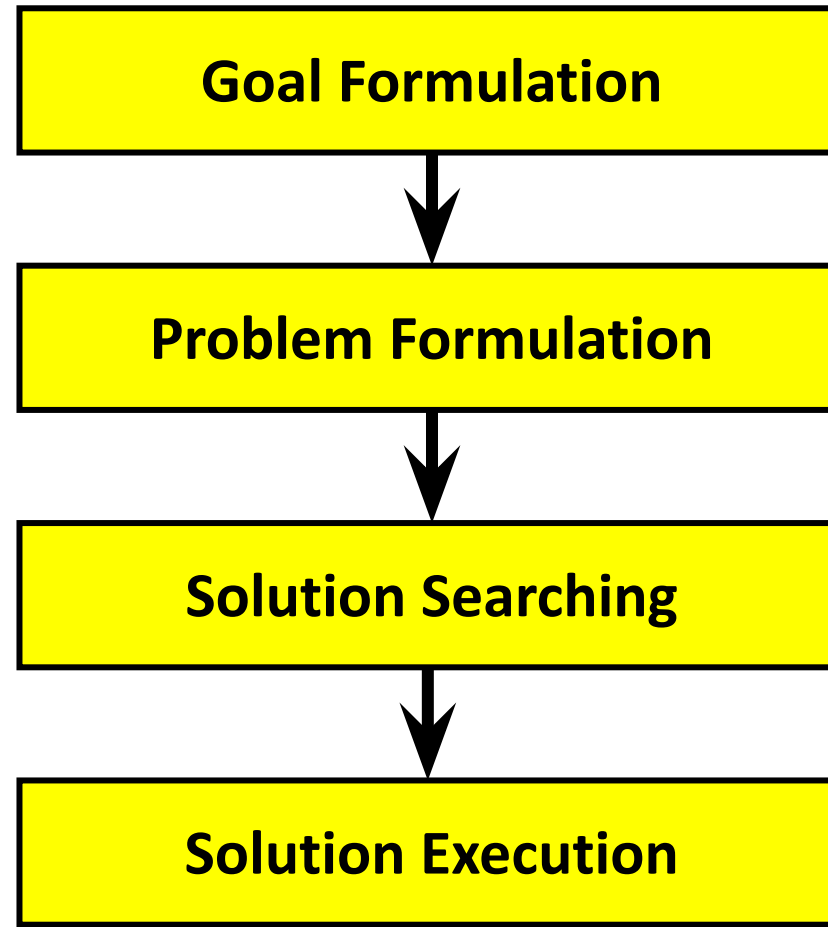


Suppose an agent is in the city of Arad enjoying a touring holiday. The agent **wishes to have different kinds of wonderful experiences in Romania**, and it has a non-refundable ticket to fly out to Bucharest the following day...

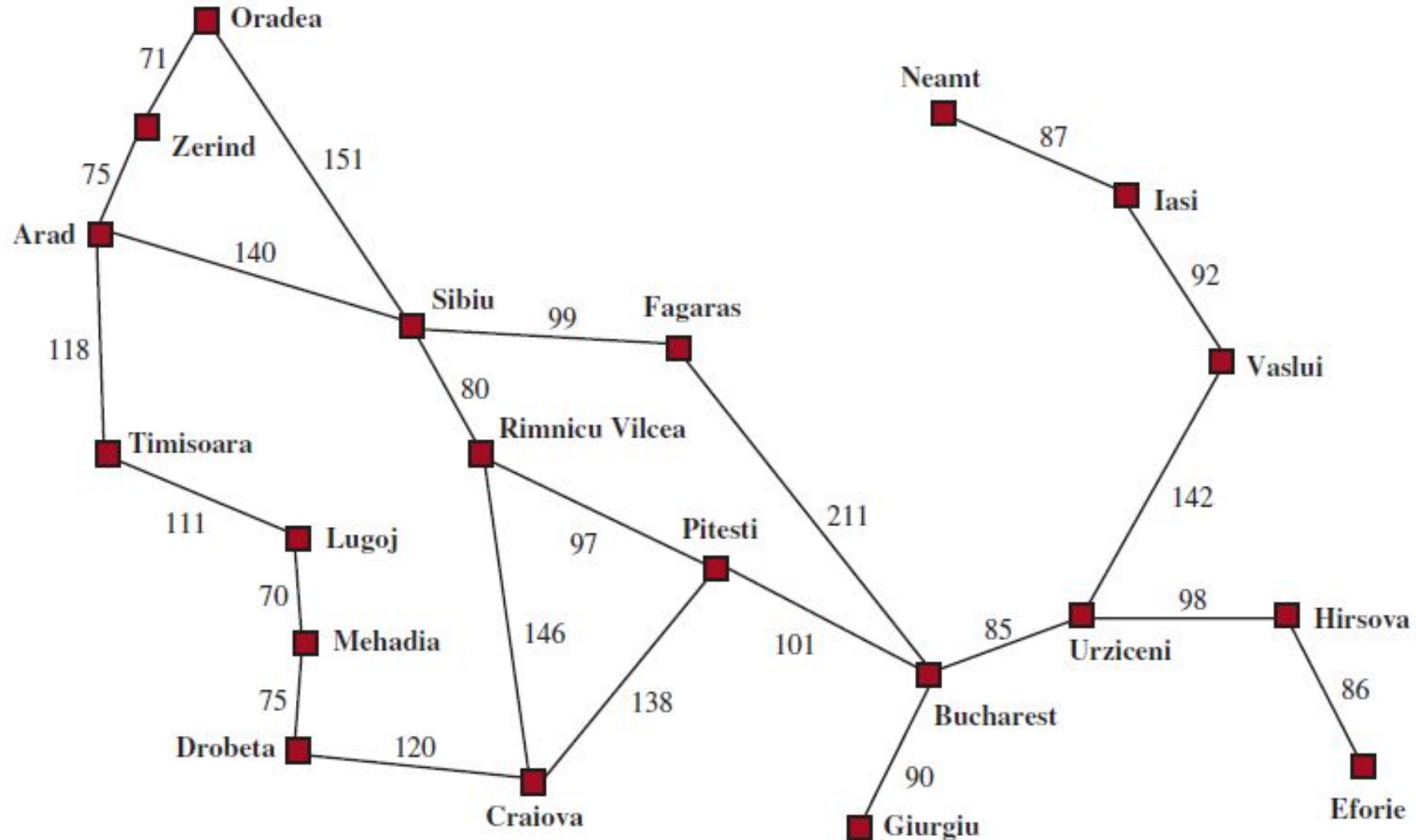


- What is the agent's goal?
- What will happen if the agent adopts the goal of getting to Bucharest?
- What should the agent do?

Problem Solving Phases



A Simplified Road Map of Part of Romania



Goal Formulation

- **Goal formulation** is the first step in problem solving.
- An agent's **performance measure** may contain many factors, goals can:
 - Limit the objectives that the agent is trying to achieve.
 - Limit the actions it needs to consider.
 - E.g., **Goal: Driving to Bucharest (from Arad).**
- Next step
 - What sorts of actions and states it should consider to reach a goal state?

Driving to Bucharest: Scenario #1

□ Problem formulation

□ Scenario #1

- The agent is unfamiliar with the geography of Romania.
- Unaware of Arad is connected to Sibiu, Timisoara, and Zerind
- Unsure which action to be taken as it lacks sufficient information about the resulting states.

Driving to Bucharest: Scenario #2

□ Problem formulation

□ Scenario #2

- The agent has a map of Romania.
- Aware of all possible choices of action at any point of time.
- Aware of the resulting state with respect to the action taken.
- Can consider a hypothetical journey that finds a way to Bucharest.

□ Assumption

- Environments are episodic, single agent, fully observable, deterministic, static, discrete, and known.

Problem Formulation

- **Problem formulation** is the process of creating an **abstract model** of the relevant part of the world.
- A formulated problem consists of:
 - A description of the states.
 - A description of actions necessary to reach the goal.
- Example
 - State: City
 - Action: Traveling from one city to an adjacent city

Search and Execution

□ Search

- The agent simulates sequences of actions in its model.
- The simulation stops when a sequence of actions that reaches the goal is found, or it finds that no solution is possible.
- Such a sequence is known as a **solution**.

□ Execute

- The agent executes all actions in the solution, one at a time.

Driving to Bucharest: Example

- **Assumption: Properties of Environment**
 - **Fully observable:** The agent always knows the current state.
 - **Deterministic:** Each action has exactly one outcome.
 - **Discrete:** There is a finite number of states and a finite number of actions are available at any given state.
 - **Known:** The "geography" of the environment is known a priori.
- Under ideal conditions as a result of the above assumptions, the **solution** to any problem is a **fixed sequence of actions**.

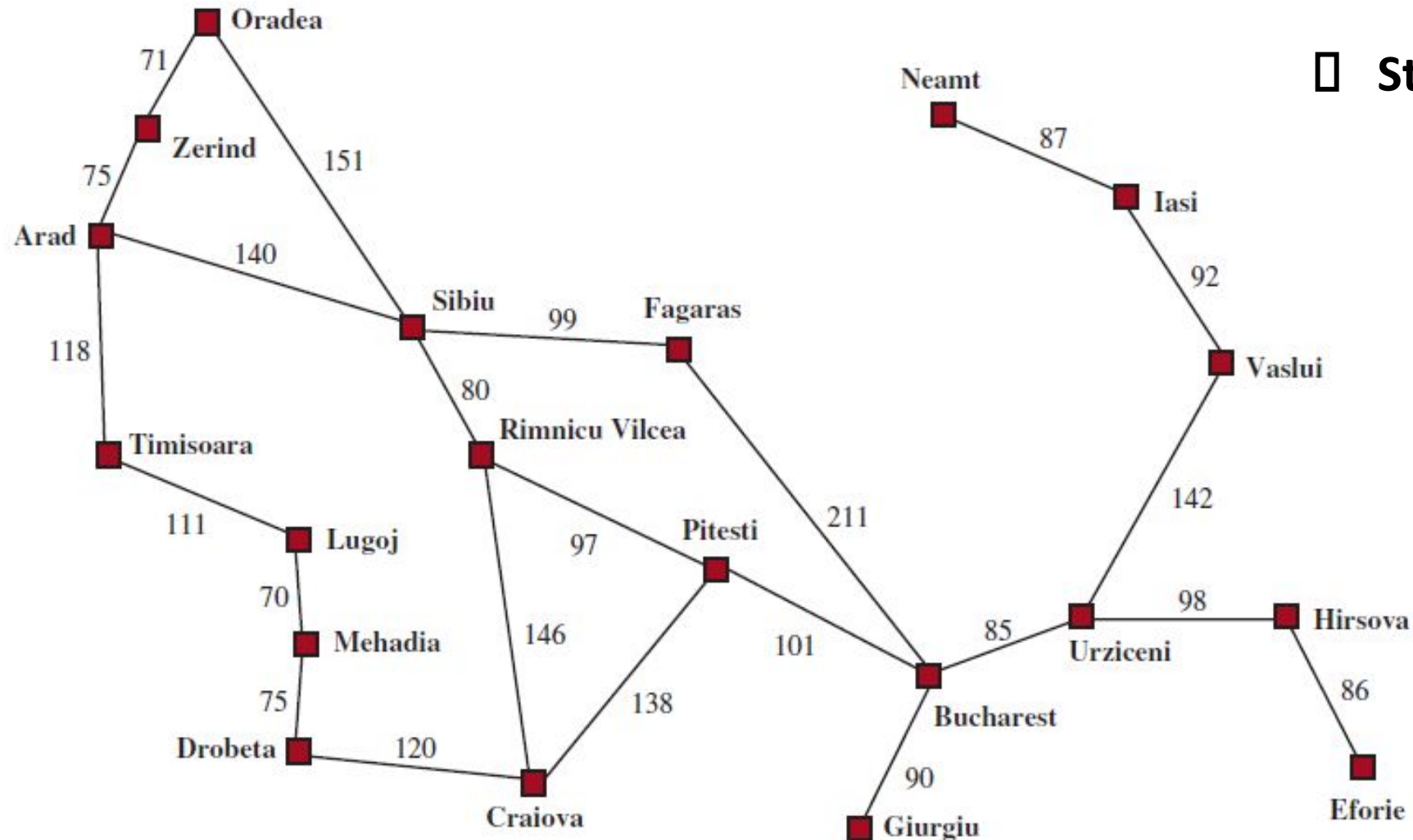
Problem Formulation

- A problem can be defined formally by 6 components:
 - States
 - Initial state
 - Goal state
 - Action
 - Transition model
 - Cost function

Problem Formulation

- A problem can be defined formally by 6 components:
 - **States:** A set of possible states that the environment can be in, referred to as **state space**.
 - **Initial state:** A state that the agent starts in.
 - **Goal state:** A state that represents or satisfies the goal.
 - **Action:** A set of possible actions available to the agent in a particular state.
 - **Transition model:** A description of what each action does.
 - **Cost function:** The cost of applying an action in state to reach another state.

State Space Graph



□ **State space** can be represented as a **graph** in which the **vertices** are states and the directed **edges** between them are actions.

Problem Formulation

- Initial state: Arad
- Goal state: Burcharest
- Action
 - $ACTIONS(s)$ returns a finite set of actions that can be executed in s .
 - $ACTION(Arad) = \{ToSibiu, ToTimisoara, ToZerind\}$
- Transition Model
 - $RESULT(s, a)$ returns a state that results from doing action a in state s .
 - $RESULT(Arad, ToZerind) = Zerind$

Problem Formulation

- **Cost function**

- **ACTION-COST(s, a, s')** returns a numeric cost of applying action **a** in state **s** to reach state **s'**.

- A problem-solving agent should use a cost function that **reflects its own performance measure**.

- **Example**

- For route-finding agents, the cost of an action might be the length in miles, or it might be the time it takes to complete the action.

Solution

- A sequence of actions forms a **path**.
- A **solution** is a path from the initial state to a goal state.
- An **optimal solution** has the lowest path cost among all solutions.

- **Assumption:** Action costs are additive, so the total cost of a path is the sum of the individual action costs.

Abstraction

- The formulated problem is a **simplified representation of the real world**.
- The process of **removing details from a representation** is called **abstraction**.
 - State abstraction
 - The starting location (in Arad): **Arad**
 - Action abstraction
 - The driving action (to Sibiu): **ToSibiu**

Abstraction

- An abstraction is **valid** if any abstract solution can be elaborated into a solution in the more detailed world.
- An abstraction is **useful** if carrying out each of the actions in the solution is easier than the original problem.
- **Good abstraction**
 - Removing as much detail as possible while retaining **validity**, and yet ensuring that the abstract actions are **easy to carry out**

Example Problems

□ **Standardized problems**

- Intended to illustrate or exercise various problem-solving methods.
- May have a lower complexity compared to real-world problems.
- Can be given a concise and exact description.
- A benchmark for researchers to compare the performance of algorithms.

□ **Real-world problems**

- Those problems whose solutions people actually use.
- Their formulation is idiosyncratic and not standardized.

Standardized Problem

- A **grid world** problem is a two-dimensional rectangular array of square cells in which agents can move from cell to cell.
- Typically the agent can move to any obstacle-free adjacent cell—horizontally or vertically and in some problems diagonally.
- Example
 - The Vacuum World
 - The 8-Puzzle
 - The 8-Queen

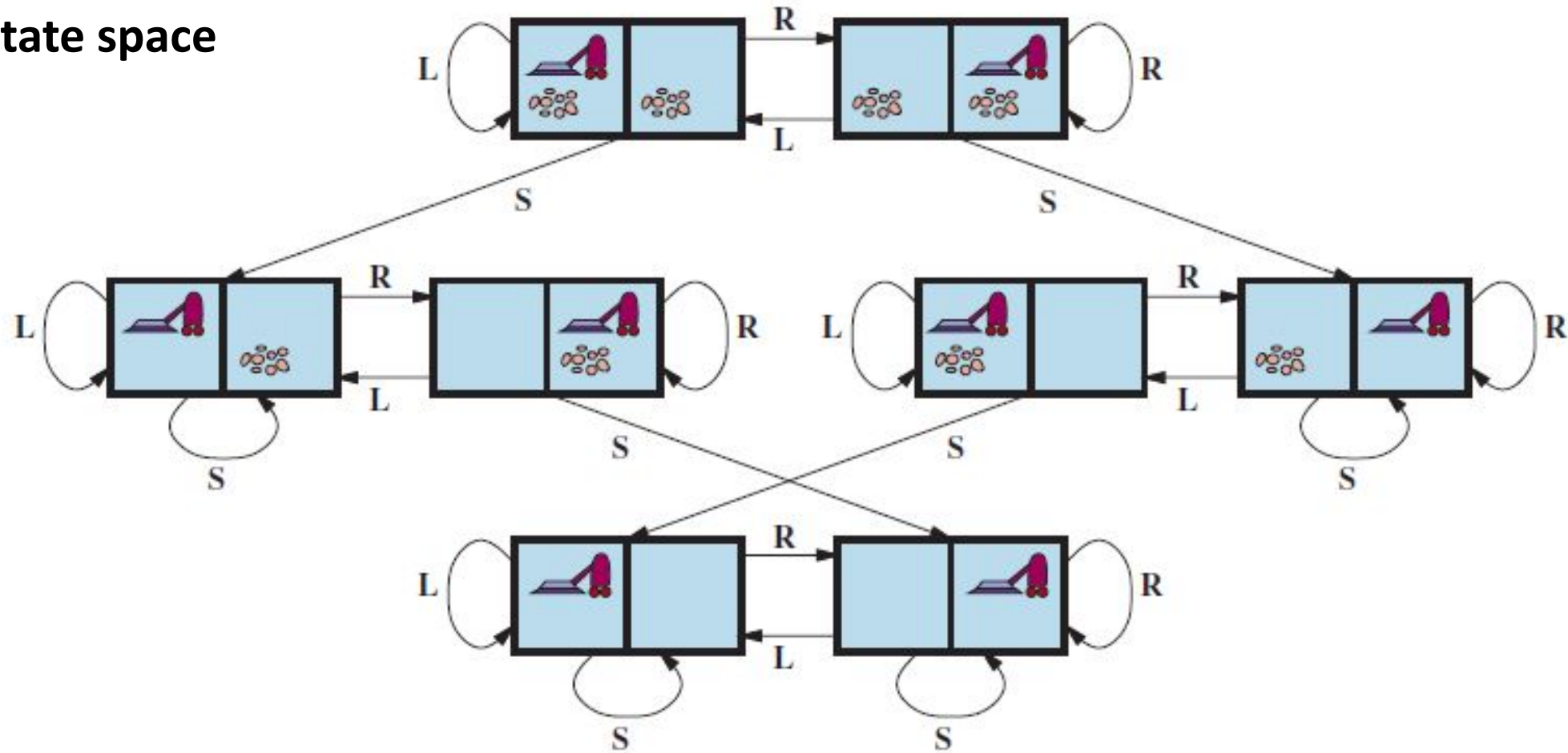
The Vacuum World

□ States

- A state of the world says which **objects** are in which cells.
- For the vacuum world, the objects are the **agent** and **any dirt**.
- Example: The 2-cell version
 - The agent can be in either of the 2 cells.
 - Each cell can either contain dirt or not.
 - There are $2 \times 2^2 = 8$ possible world states.
 - A larger environment with n locations has $n \times 2^n$ states.

The Vacuum World

□ State space



The state space for the vacuum world. Links denote actions: L = Left, R = Right, S = Suck.

The Vacuum World

□ Initial state

- Any state can be designated as the initial state.

□ Goal state

- The states in which every cell is clean.

□ Action

- Left, Right, and Suck.

□ Cost function

- Each action costs 1.

The Vacuum World

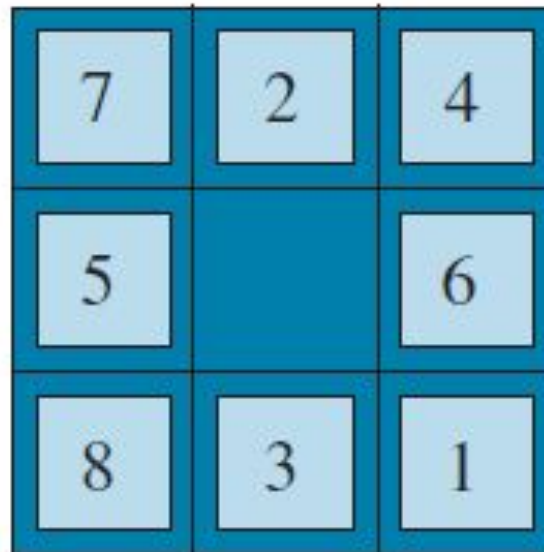
□ Transition model

- Suck removes any dirt from the agent's cell, unless the cell is clean.
- Left moves the agent to one cell on the left, unless it hits a wall.
- Right moves the agent to one cell on the right, unless it hits a wall.

The 8-Puzzle

□ States

- A state description specifies the location of the 8 tiles and the blank space.
- The 8-puzzle has $9!/2 = 181440$ reachable states
- A larger environment with n squares has $(n+1)!/2$ reachable states



The 8-Puzzle

□ Initial state

- Any state can be designated as the initial state.

□ Goal state

- Any state can be designated as the goal state.

□ Action

- Movements of the blank space: Left, Right, Up, or Down.

□ Cost function

- Each action costs 1.

7	2	4
5		6
8	3	1

Start State

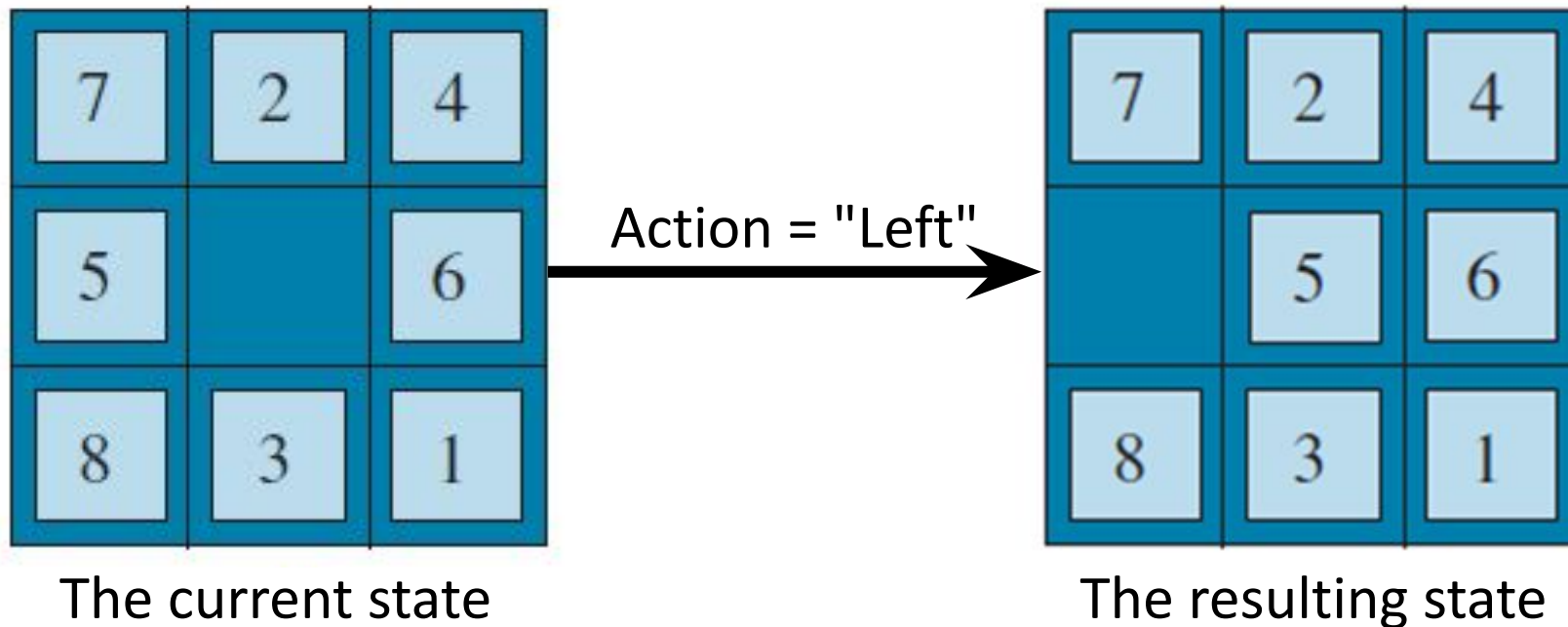
	1	2
3	4	5
6	7	8

Goal State

The 8-Puzzle

□ Transition model

- It maps a state and action to a resulting state, for example, the action "**Left**" is applied to the cuurent state in the figure below.



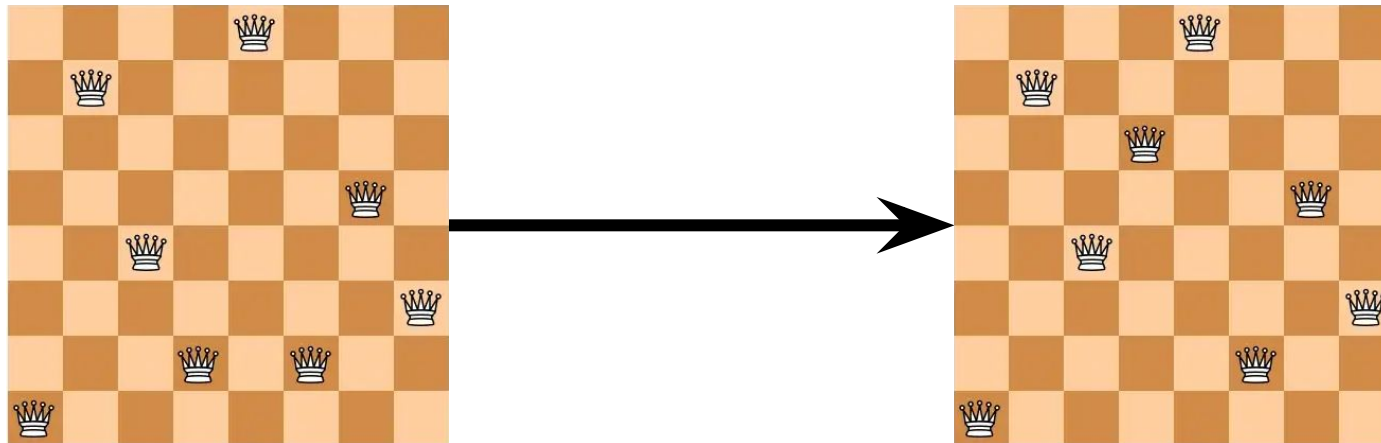
The 8-Queens

□ Incremental formulation

- It starts with an empty state and each action adds a queen to the state.

□ Complete-state formulation

- It starts with all the 8 queens on the board and moves one queen at a time.



The 8-Queens

□ States (version 1)

□ A state description specifies the arrangement of 8 queens on the board.

□ Possible states

$$64 \times 63 \times 62 \times 61 \times 60 \times 59 \times 58 \times 57 \approx 1.8 \times 10^{14}$$

□ States (version 2)

□ A state description specifies the arrangement of 8 queens with only one queen per column on the board.

□ Possible states

$$8^8 \approx 1.7 \times 10^7$$

The 8-Queens

□ Initial state

- Any state can be designated as the initial state.

□ Goal state

- A state in which all the 8 queens are none attacked (i.e., vertically, horizontally and diagonally) on the board.

□ Action

- Move any queen vertically, either "Up" or "Down", to any empty square.

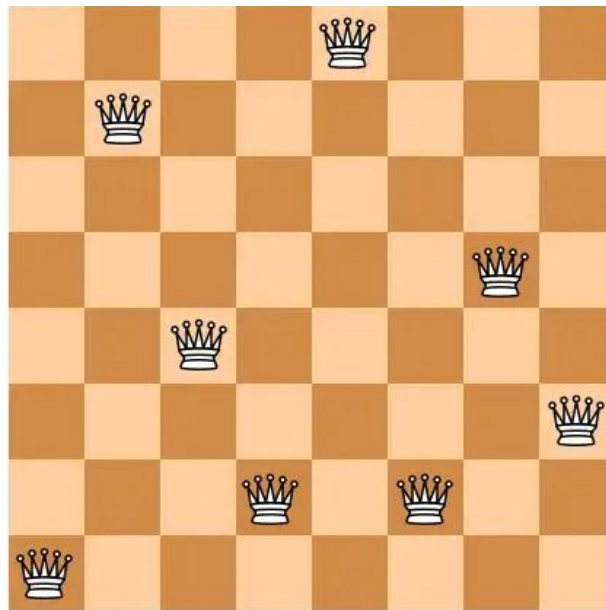
□ Cost function

- Each action costs 1.

The 8-Queens

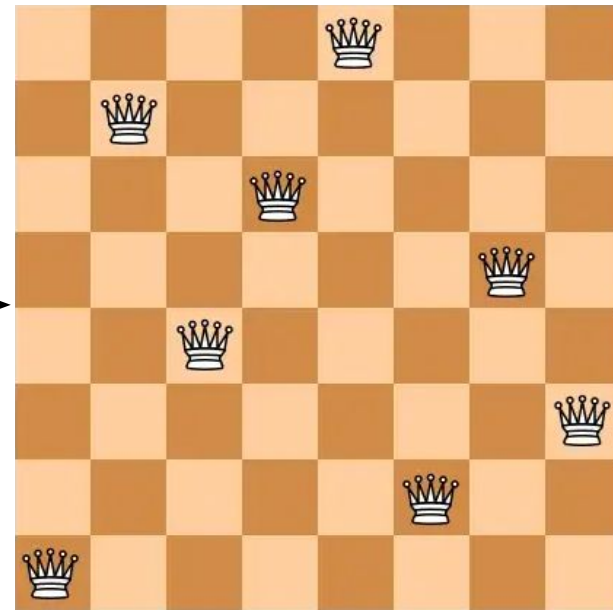
□ Transition model

- It maps a state and action to a resulting state, for example, the action "Up" is applied onto a queen in the 4th column.



The current state

Action = "Up"



The resulting state

Real-World Problem

- The **route finding problem** of traveling from Arad to Burcharest is a standardized problem; and an extension of it is a real-world problem of providing driving direction by the GPS application such as Waze.
- Examples
 - The VLSI layout problem
 - Traveling salesperson problem
 - Robot navigation
 - Automatic assembly sequencing

Airline Travel Problems

□ States

- Each state obviously includes a location (e.g., an airport) and the flight time.

□ Initial state

- Any airport with a flight time.
- E.g., The user's home airport - KLIA airport @8am.

□ Goal state

- Any airport with a flight time.
- E.g., A destination city - Senai airport @8pm.

□ Action

- Take any flight from the current location at the flight time.

Airline Travel Problems

□ **Transition model**

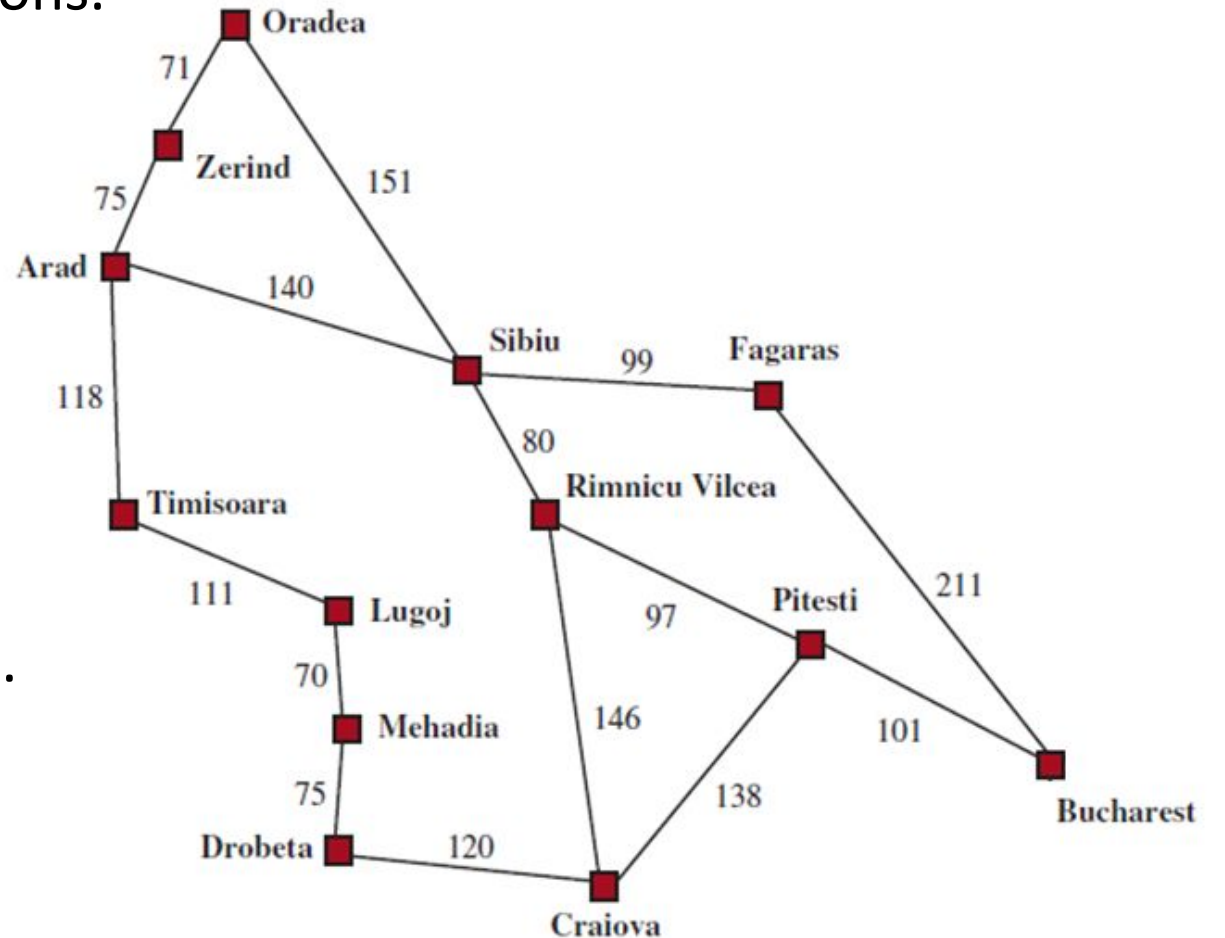
- The state resulting from taking a flight will have the flight's destination as the new location and the flight's arrival time as the new time.

□ **Cost function**

- A combination of monetary cost and flight time.
- It may take into consideration waiting time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.

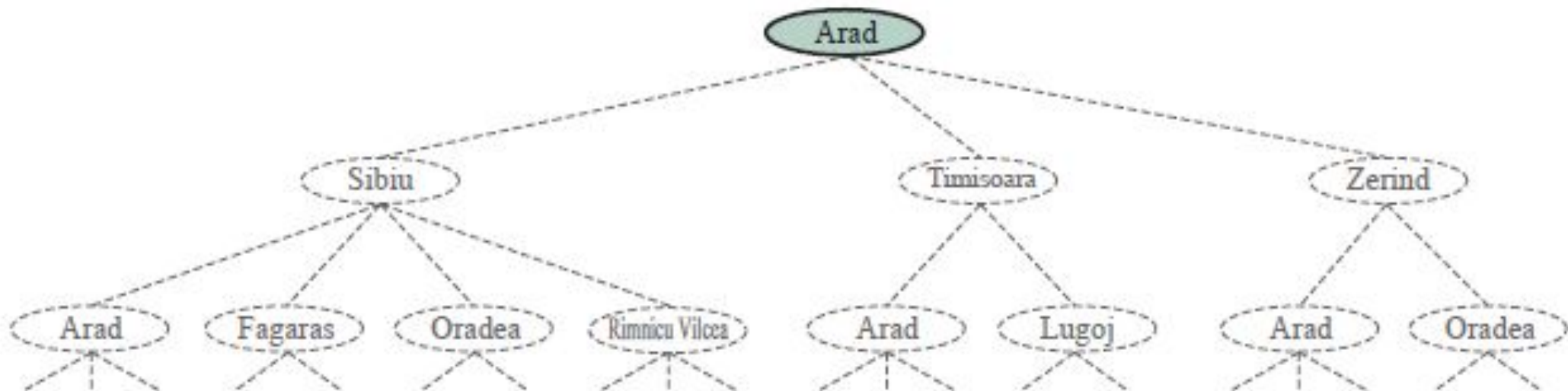
Searching for Solution

- A **solution** is a fixed sequence of actions.
- A **search algorithm** takes a search problem as input and returns a solution, or an indication of failure.
- A **search tree** is a representation used to illustrate a search simulation.



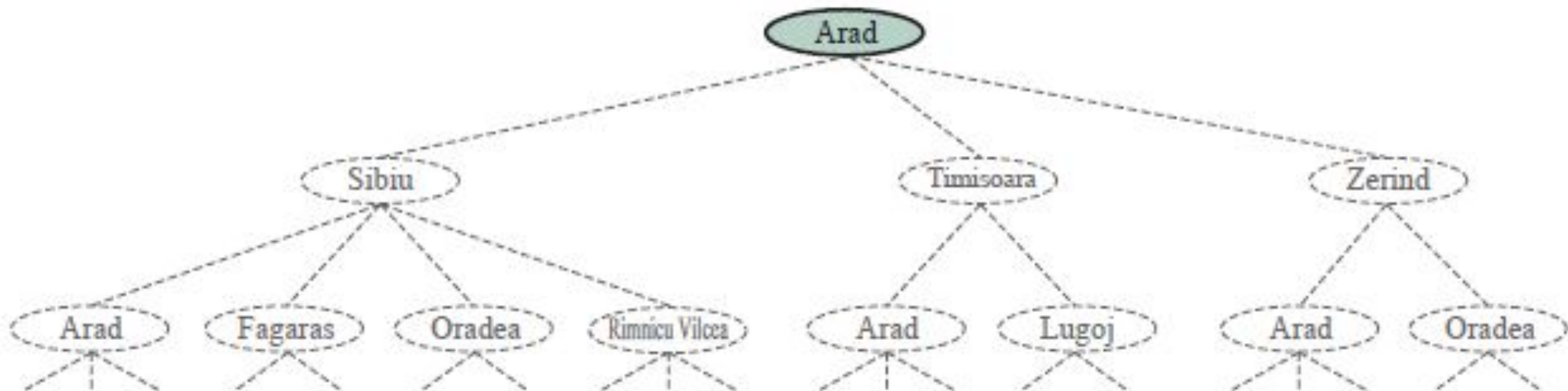
Search Tree

- A **search tree** consists of
 - **Node**: Corresponds to a state in a **state space**
 - **Edge**: Action
- The **root** of the tree corresponds to the **initial state** of the problem.
- Initial state: **Arad**



How A Search Tree Works?

- **Frontier** (or **open list**) refers to a set of **unexpanded nodes** reside in the memory.
- **First step:**
 - Root node is added to the frontier.
 - Then, it is removed from the frontier for **expansion**.

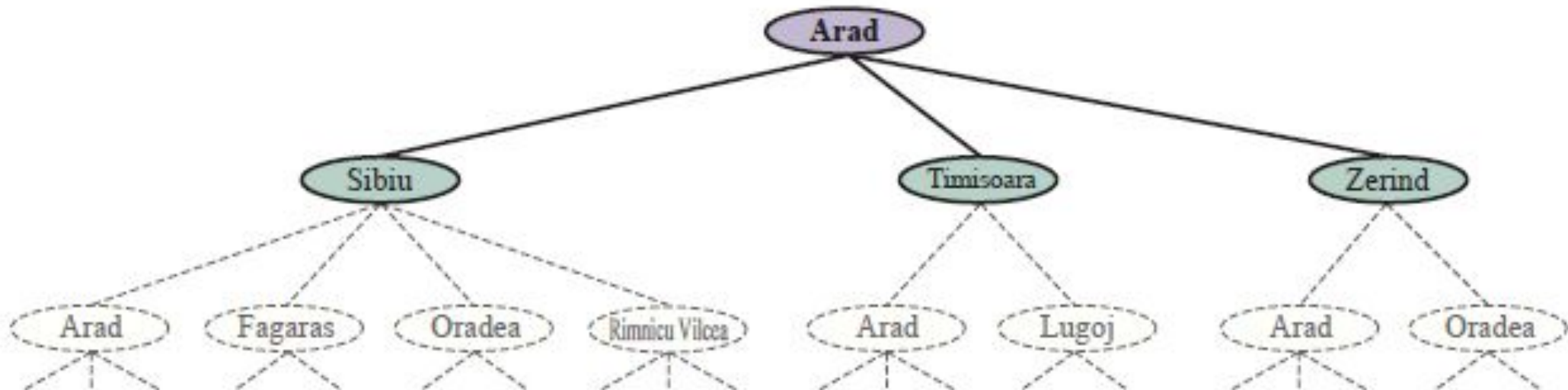


How A Search Tree Works?

- **Node expansion** refers to the application of available action(s) onto a node.
- **Next step:**
 - Apply all actions available to the current node to generate a new set of states.
 - E.g., $\text{ACTIONS}(\text{Arad}) = \{\text{ToSibiu}, \text{ToTimisoara}, \text{ToZerind}\}$
 - $\text{RESULT}(\text{Arad}, \text{ToSibiu}) = \text{Sibiu}$
 - $\text{RESULT}(\text{Arad}, \text{ToTimisoara}) = \text{Timisoara}$
 - $\text{RESULT}(\text{Arad}, \text{ToZerind}) = \text{Zerind}$

How A Search Tree Works?

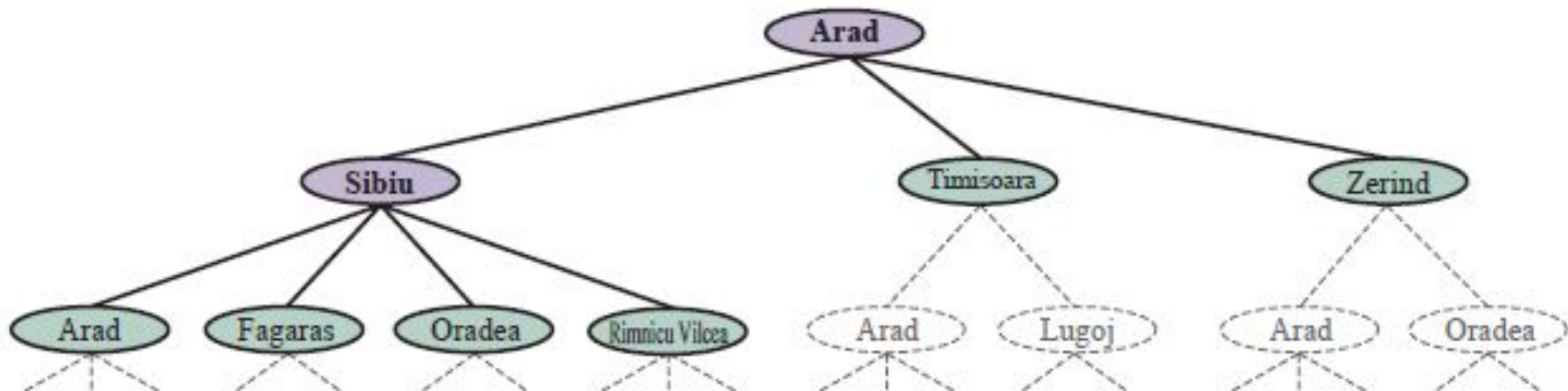
- **Node generation** means the generation of new nodes (also referred to as **child nodes** or **successors**) and then add them to the frontier.
 - 3 new successors: **Sibiu, Timisoara, Zerind**
- **Question:** Which node should be expanded?



How A Search Tree Works?

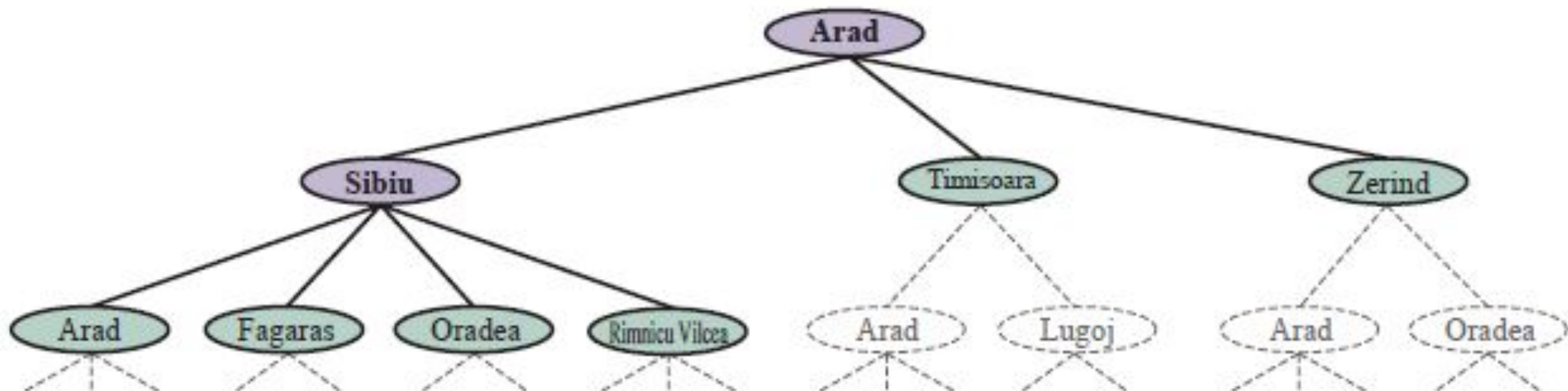
□ Next step:

- Suppose Sibiu is chosen... run a goal test on **Sibiu**, and then expand it to get four successors.
- Expansion on the frontier continues until either **a solution is found** or **there are no more states to expand**.

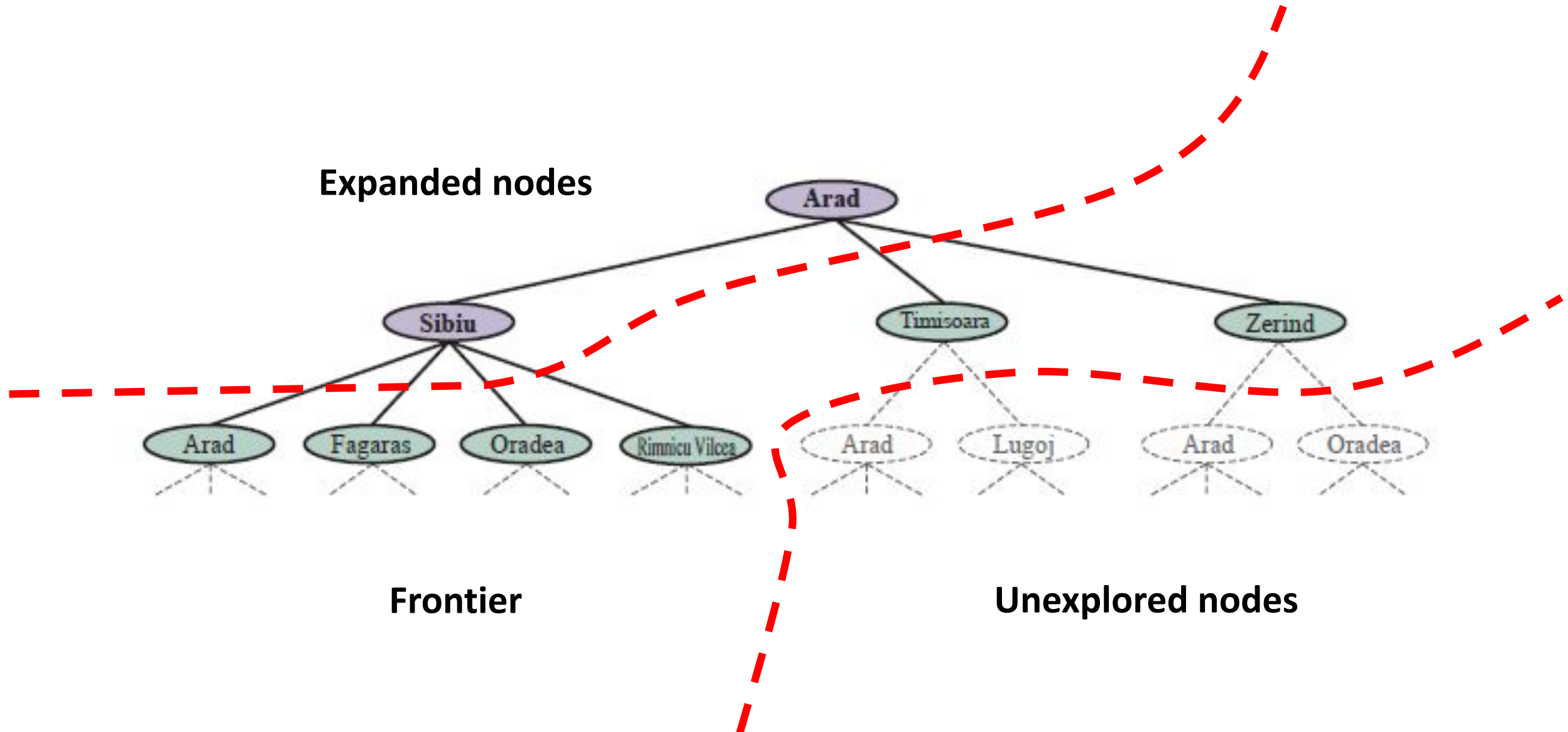


How A Search Tree Works?

- Any state that has had a node generated (whether or not that node has been expanded) for it has been **reached**.
- Nodes that have already been expanded are referred to as the **explored set** (or **closed list**).

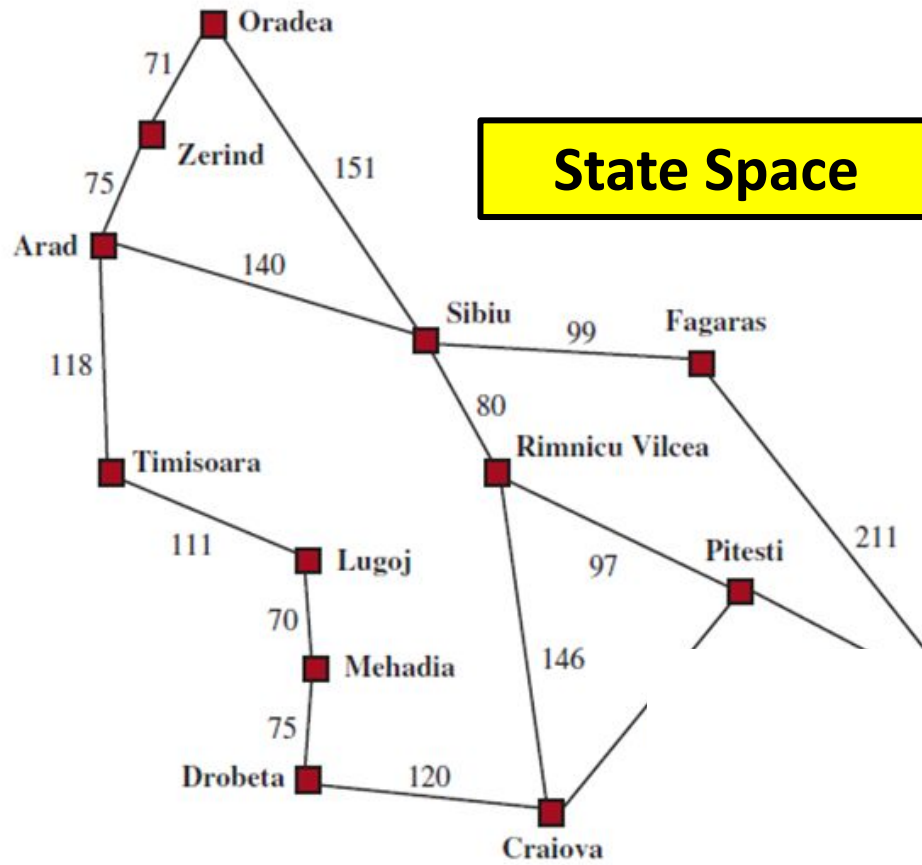


How A Search Tree Works?

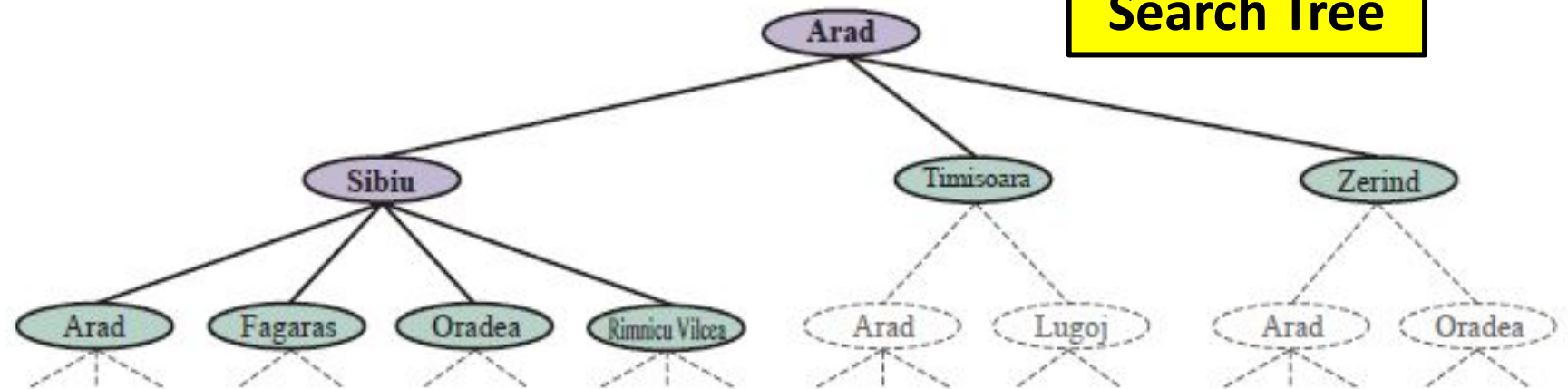


Search Tree vs. State Space

State Space

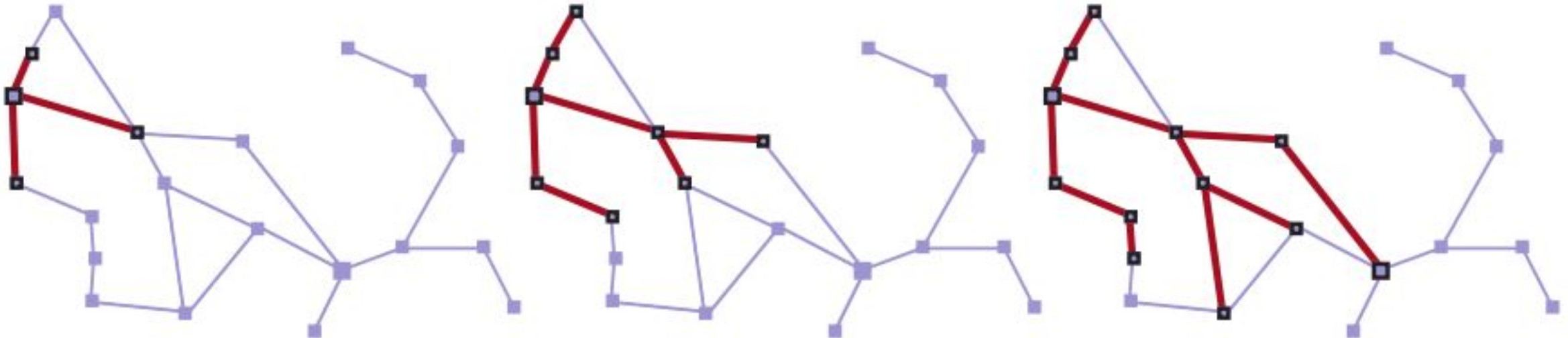
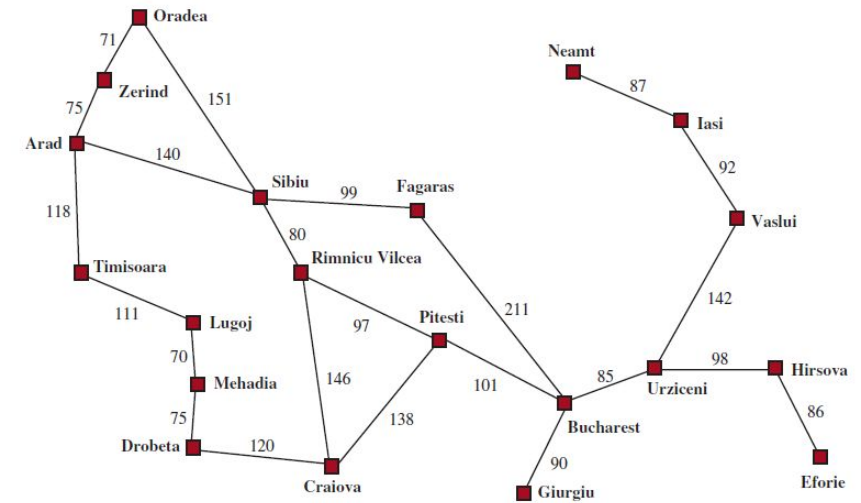


Search Tree



Search Tree vs. State Space

- Search algorithms superimpose a **search tree** over the **state-space graph** during the search simulation.
- **Node expansion:** Arad, Timisoara, Sibiu, Zerind, Lugoj, Rimnicu Vilcea, fagaras, Oradea, etc.



Search Tree vs. State Space

□ State space

- It describes the set of states in the world, and the actions that allow transitions from one state to another.
- Each state is uniquely represented in a state space.

□ Search tree

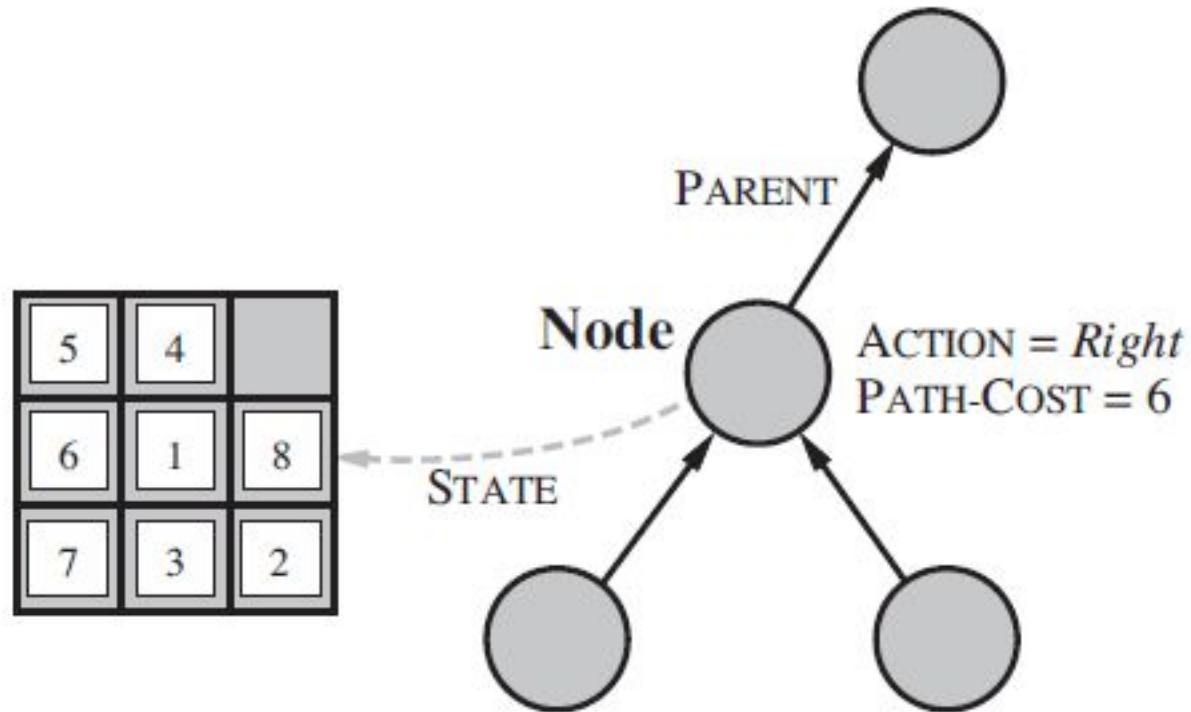
- It describes paths between these states, beginning from the initial state reaching towards the goal state.
- Multiple nodes correspond to the same state can exist in a search tree.

Search Data Structure: Nodes

- Search algorithms require a **data structure** to keep track of the search tree.
- **Data structures** refer to structured ways of organizing, managing, and storing data **for the purpose of efficient access to data**.
- A **node** in the tree is represented by a data structure with four components:
 - **n.STATE**: The state to which the node corresponds.
 - **n.PARENT**: The node in the search tree that generated this node.
 - **n.ACTION**: The action that was applied to the parent to generate the node.
 - **n.PATH-COST**: The total cost of the path from the initial state to the node.

Search Data Structures: Nodes

- Following the **PARENT** pointers backwards from a node to the root allows us to recover the states and actions along the path to that node.



Search Data Structures: Frontier

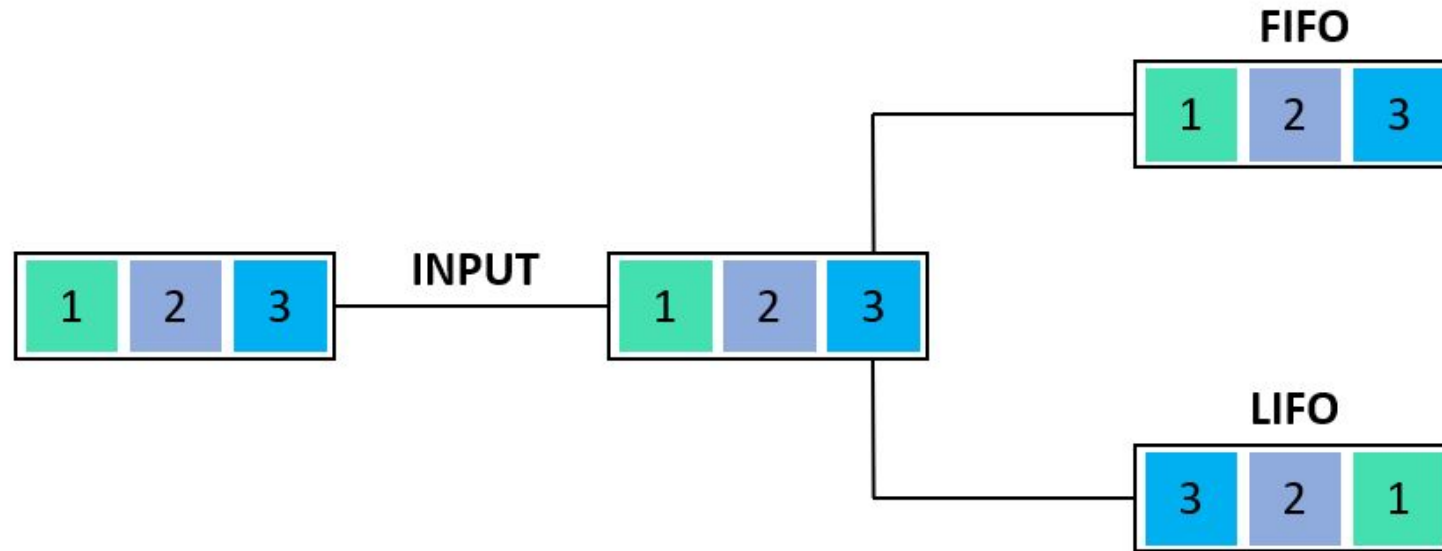
- **Frontier** needs to be stored in a way such that the next node can be **easily chosen for expansion** according to its **preferred strategy**.
- The appropriate data structure for this is a **queue** because the following operations need to be performed on a frontier:
 - **IS-EMPTY(frontier)**: Returns true only if there are no nodes in the frontier.
 - **POP(frontier)**: Removes the top node from the frontier and returns it.
 - **ADD(node, frontier)**: Inserts a node into its proper place in the frontier.

Search Data Structures: Frontier

- Three kinds of **queues** are used in search algorithms:
 - **Priority queue**
 - **First-in-first-out (FIFO) queue**
 - **Last-in-first-out (LIFO) queue (also known as a stack)**
- These queues are characterized by the order in which they store the inserted nodes.

FIFO Queue and LIFO Queue

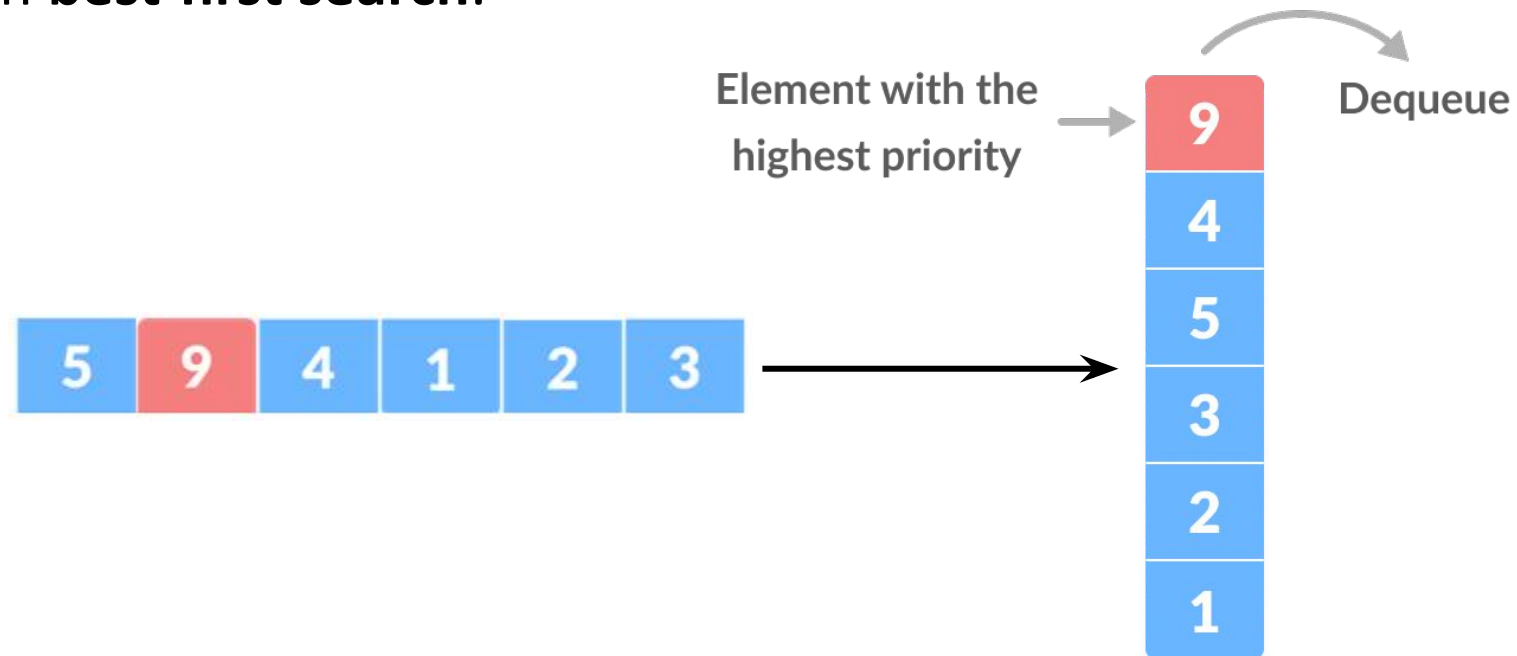
- The **FIFO queue** first pops the node that was added to the queue first.
- It is used



- The **LIFO queue** first pops the most recently added node.
- It is used in **depth-first search**.

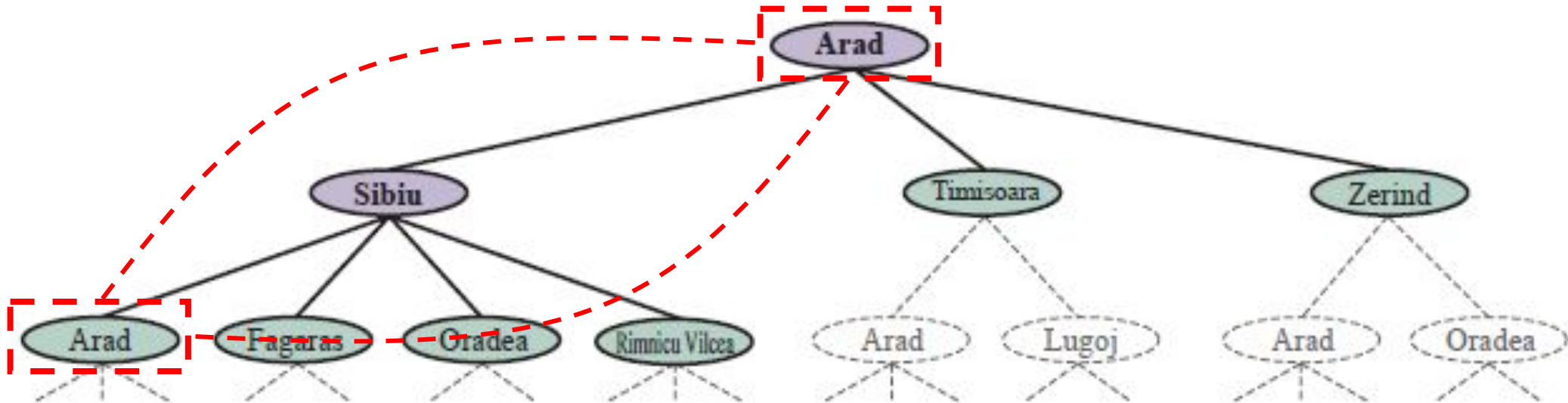
Priority Queue

- The **priority queue** first pops the node with the **minimum cost** according to some **evaluation functions, f** .
- It is used in **best-first search**.



Loopy Paths

- The search tree contains a **repeated state**.



- Any path that contains a repeated state is known as a **loopy path** (or **cycle**).
- A loopy path can make a search algorithm stuck in an **infinite loop**.

Redundant Paths

□ **Problem:** Travel from Arad to Sibiu.

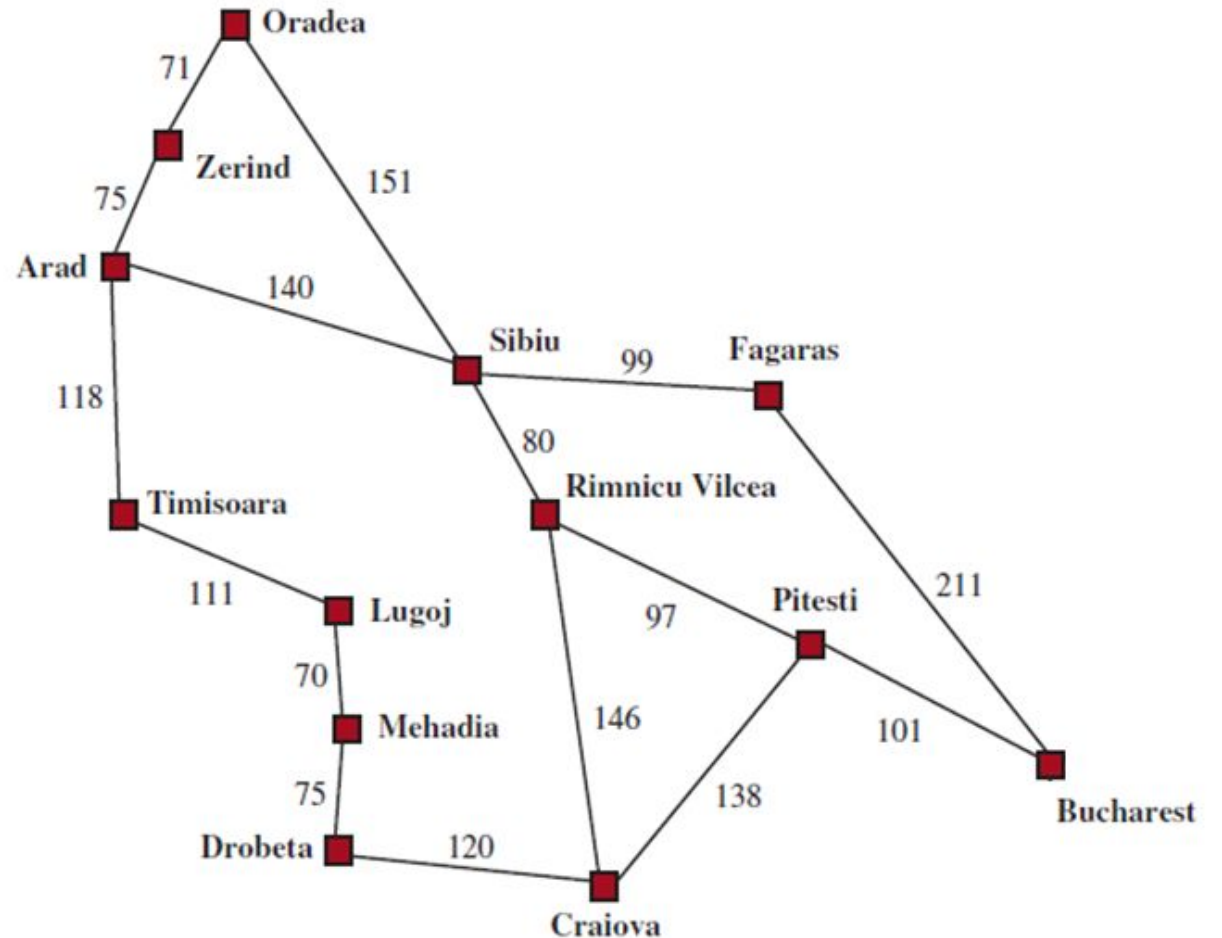
□ **Solutions**

□ A: Arad-Sibiu

□ B: Arad-Zerind-Oradea-Sibiu

□ Since A is a better way to get to the same state than B does, so B is a **redundant path**.

□ A loopy path is a special case of **redundant path**.



Solutions to Redundant Paths

- To avoid exploring redundant paths, one has to remember where one has been.
- Solutions
 - A **graph search algorithm** checks for redundant paths via the use of **reached**.
 - A **special implementation** that follows up a path from the current node up until the root node can detect the presence of a loopy path.
 - A **tree-like search algorithm** (or **tree search algorithm**) does not check for redundant paths because it doesn't keep track of **reached**.

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 3: Solving Problems by Searching (page 81 to page 93)

The End